

NVIDIA JETSON THOR

GLOBANT PHYSICAL AI HACKATHON

Talk to your Company

A live 3D twin of your plants that you can talk to.

Yash Kavaiya

Globant Physical AI Hackathon • October 2026

F3



F2



SECTION 1

The problem

01

Nobody can watch every camera



Events go unseen

Near-misses, restricted zones and missing helmets slip by.



Answers are slow

"What is happening on floor 3?" means walking there.



Walls are mute

Camera walls show video. They cannot be asked anything.



Cloud is a poor fit

Factory video off site is slow, costly and often not allowed.

The manager needs **one view of the whole site**, and **a way to ask it things**.

A live 3D twin of your company that you can talk to.

Ask by voice. The twin flies to the right floor, shows what the cameras see, and answers out loud.

SECTION 2

What it does

02



The whole company, live in one view.

Every person and vehicle the cameras see is a live marker on the right floor.

WHAT IT DOES

You ask in plain language, by voice or by text

YOU ASK

“Give me a status of both plants.”

YOU ASK

“Show me Plant 2, floor 3.”

YOU ASK

“Any safety issues in the last ten minutes?”



Answers are spoken, shown as text, and move the twin.

Four rules turn movement into safety events



Near miss

Person within 2 m of a closing forklift.



Restricted zone

Person inside for more than 2 seconds.



No helmet

The vision check on a person says no.



Crowding

More than 4 people within 3 m.

Events land on the **right floor**, are stored, and can be asked about by plant, floor, type and time.

WHAT IT DOES

Every floor has a live camera, with detections boxed



Focus a floor, see its camera

The view opens beside the twin.



One detection, two views

What places the marker also draws the box.



Nothing extra is exposed

One stream at a time, on the one app port.

Feed any camera from a recording or a webcam



Recorded video

mp4 files, looped as live streams.



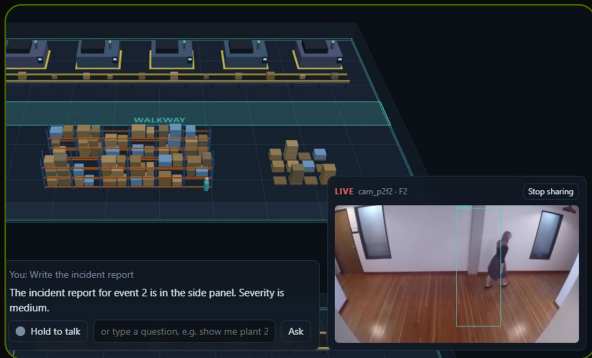
A webcam

Shared from the browser, or attached to the machine.



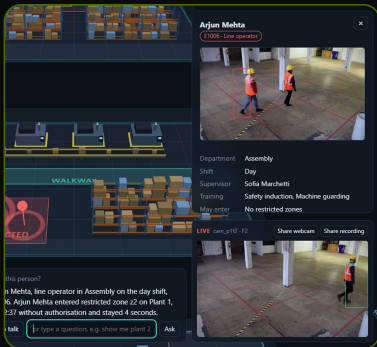
Share recording

Pick any video file; it is analysed on the spot.



WHAT IT DOES

The alert names the person. Ask who it is.



Who

Name, role, shift, supervisor and training.



When and how long

Entered, alert raised, left, time in the zone.



Allowed or not

Authorisation, and other incidents today.

SYNTHETIC EMPLOYEES

SIMULATED BADGE FEED

NO FACE RECOGNITION

One more sentence writes the incident report

YOU ASK

“Write the incident report.”



Complete

What, where, when, who, severity, action.



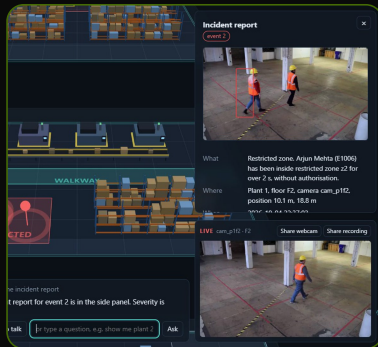
Grounded

Filled from the stored event, nothing invented.



Evidence attached

The camera snapshot from that moment.



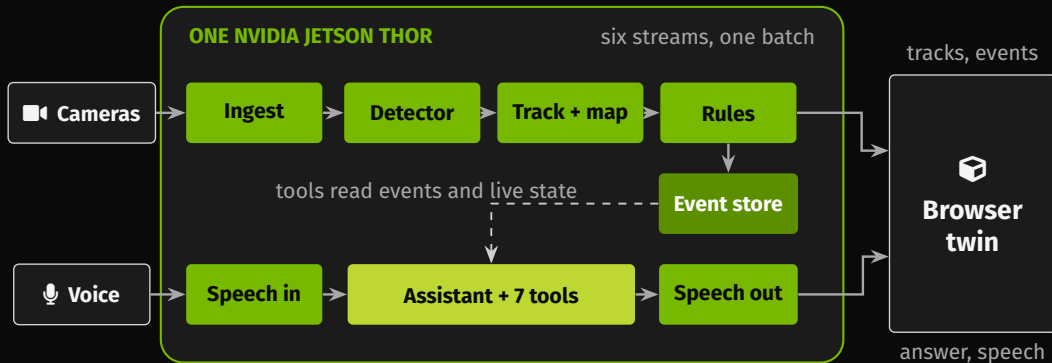
SECTION 3

How it works

03

HOW IT WORKS

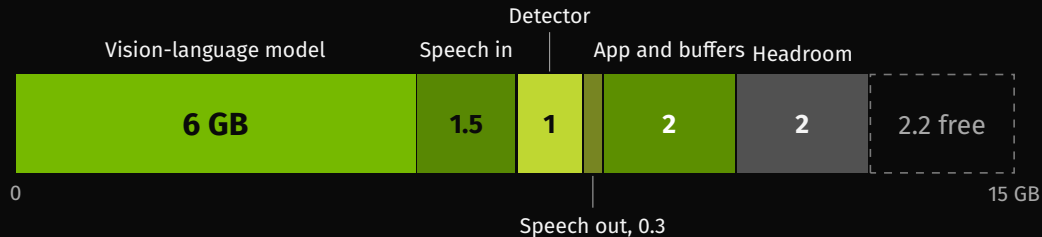
One device takes video to a spoken answer



One vision-language model serves the conversation and the “look at this frame” tool.

HOW IT WORKS

The planned model stack fits inside 15 GB



Vision model

3B, half precision.



Whisper and Piper

Speech stays local.



YOLO on TensorRT

One batch, six streams.

Planned budget. It still has to be measured on the Jetson.

Every heavy component has a mock



Perception

REAL

Video and
detector

MOCK

Simulated
people



Assistant

REAL

Vision model

MOCK

Keyword router



Speech in

REAL

Whisper

MOCK

A fixed question



Speech out

REAL

Piper

MOCK

A chime

A backend that fails to load falls back to its mock, so the demo cannot die.

SECTION 4

Architecture in detail

04

One process runs three loops at once



Perception threads

One thread per stream decodes video. One batched detector, then tracker and mapper.



Broadcast loop, 5 Hz

Runs the rules on the latest tracks, stores events, sends state to the browser.



Agent loop, per question

Speech to text, up to three rounds of tool calls, then the spoken answer.

One FastAPI process on 0.0.0.0:8000. The browser is the only client; that is the only port.

Perception: five stages from pixels to floor metres

STAGE	MODULE	WHAT IT DOES
Ingest	<code>ingest.py</code>	GStreamer hardware decode, OpenCV fallback. Files, a webcam or shared frames.
Detect	<code>detector.py</code>	YOLO on TensorRT FP16. Every stream's latest frame in one batch.
Track	<code>tracker.py</code>	ByteTrack-style association per stream. Gives each person a stable id.
Map	<code>mapper.py</code>	Foot point through the camera homography to floor metres.
Mock	<code>mock.py</code>	Simulated people; a scripted scenario every 30 s raises a real event.

Rules are pure functions with fixed thresholds

EVENT	RAISED WHEN	NOTES
near_miss	Person and forklift within 2 m, closing above 0.5 m/s.	Severity 3.
restricted_zone	Person inside a restricted polygon for over 2 s.	Once per stay; authorised people exempt.
no_helmet	The vision model says no about a person crop.	One unchecked person every 3 s.
crowding	More than 4 people within a 3 m radius.	Severity 1.

Unit-tested, debounced for 30 seconds, and the same on mock and real tracks.

Tracks live in memory. Events live in SQLite.

TRACK, IN MEMORY

id, zone, cls
x, y, vx, vy
bbox, last_seen

Positions in floor metres, sent
five times a second.

EVENT, ONE ROW IN SQLITE

id, ts, plant, floor, camera_id
type, severity, track_ids
x, y, snapshot_path, summary, person_id

Queried by plant, floor, type and time. Open for ten
minutes.

The report is **filled from the event row**, so it cannot contain invented facts.

One WebSocket carries eleven message types

SERVER TO BROWSER

site	the whole site, on connect
state	every track per floor, 5 Hz
event	one event row, when a rule fires
ui	focus, highlight or open a panel
transcript	what was heard or typed
answer	one sentence per message
audio	that sentence, spoken
metrics	fps, latency, GPU, memory, 1 Hz

BROWSER TO SERVER

audio_query	16 kHz WAV
text_query	typed question
select_event	marker click

PLAIN HTTP

/api/health
/api/camera/{plant}/{floor}

The assistant must call a tool before it states a fact

1

Question,
spoken
or typed

2

Model sees the
tool schemas

3

Tool runs,
twin reacts

4

Up to three
rounds

5

Final answer,
two sentences

6

Spoken,
sentence by
sentence



JSON tool calls

Parsed by the app, so
any model can be
used.



Short memory

Last six turns, current
view, last event.



Honest

"I can't see that" when
tools return nothing.

Seven tools connect the model to the site

TOOL

query_events

live_state

look

focus_view

show_event

identify_person

write_report

RETURNS

Events by plant, floor, type and time.

People, vehicles and open events now.

The vision model's answer on the frame.

Nothing.

Nothing.

Employee, visit timing, incidents.

What, where, when, who, severity, action.

EFFECT ON THE TWIN

Highlights their markers.

None.

Focuses that floor.

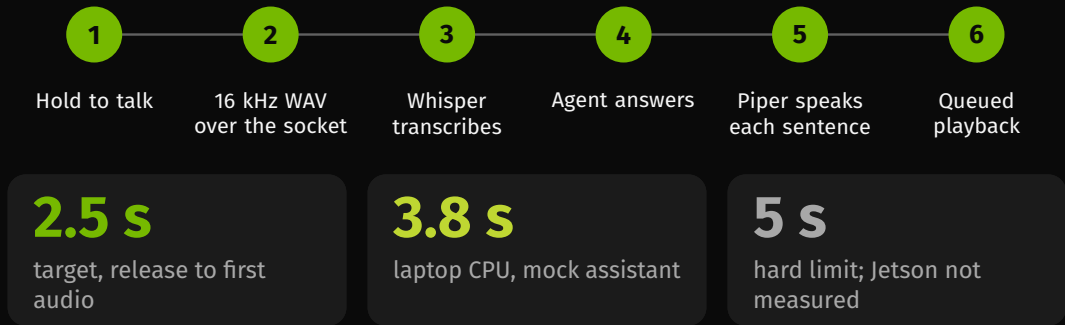
Flies the camera there.

Opens the event panel.

Opens the person panel.

Opens the report panel.

Voice: 2.5 seconds to the first spoken word



Identity comes from a badge feed, never from a face



Synthetic directory

Fictional employees and their zones.



Simulated badge feed

Gives each tracked person a badge.



One class to replace

A real system plugs in at BadgeFeed.



Authorised means silent

Allowed in the zone: no event.



Visit timing

Entered, alerted, left, how long.



Prompt rule

No naming from appearance.

The frontend is plain ES modules and Three.js

MODULE

`twin.js`, `fixtures.js`

`tracks.js`

`events.js`

`camera.js`

`voice.js`, `wave.js`

`hud.js`

`scene.js`, `ws.js`

ROLE

Plants, floors, equipment and zones from the site message.

Instanced markers, interpolated between state messages.

Pulsing event markers, click picking, the side panel.

Camera view of the focused floor; share a webcam or recording.

Push-to-talk, queued playback, the waveform.

Per-stream fps, latency, GPU, memory, the privacy line.

Renderer and camera tweens; the one WebSocket.

Three scripts take a bare Jetson to a running demo

1

setup_jetson.sh

Once. Packages, models, TensorRT export, videos.

2

preflight.sh

Every start. CUDA, decoder, disk, port, models, videos.

3

run.sh

Every start. All backends real, offline, port 8000.

NO SUDO, APT OR DOCKER

TORCH NEVER TOUCHED

8 CORES, 15 GB

Written and reviewed. Not yet run on the device.

Built spec first, and tested without a GPU

HOW THE WORK IS ORGANISED



spec.md says what

User stories with acceptance criteria.



plan.md says how

Modules, protocol, memory budget.



tasks.md says in what order

One task at a time.

HOW IT IS CHECKED



77 automated tests

Mock mode only, no GPU needed.



Tool-calling test set

15 questions; 13 must pass.



Golden path

Three clean runs on the Jetson.

Known risks, and what is already in place for each

RISK

The shared GPU slows everything.

A dependency replaces torch.

A small model is weak at tool calling.

The microphone is blocked in the browser.

A model fails to load on stage.

MITIGATION

Detector at 10 fps, models pre-warmed, a backup video.

Every install is dry-run checked first.

JSON-constrained calls; keyword router as fallback.

The text box does the same thing.

That backend starts as its mock.

SECTION 5

Status and demo

05

Proven on a laptop. The Jetson run is the next step.

77

automated tests passing

6

camera streams at once,
on a laptop CPU

0

runs on a Jetson so far

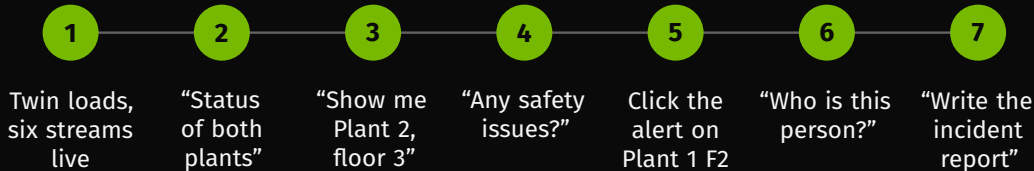
SHOWN WORKING

- ✓ Twin, events, conversation, report
- ✓ Real detection and speech on CPU
- ✓ Webcam and shared recording

STILL TO PROVE ON THE JETSON

- ⌚ Hardware decode and TensorRT
- ⌚ 3B model picks tools, 13 of 15
- ⌚ 10 fps on 6 streams, 2.5 s voice

The demo is seven steps



Then one unscripted question from a judge.

Why it matters



Safety

Events are seen, named and written up in the moment.



Operations

One person can ask about any floor without walking to it.



Privacy and cost

Audio and video stay on one device. No cloud calls.



Fit

A new site is a new configuration file, not a new project.

Ask your company a question. It shows you, and it tells you.

Talk-to-your-Company. Globant Physical AI Hackathon, built for one NVIDIA Jetson Thor.